

Síntesis, traducción y verificación de sistemas de ecuaciones utilizados en Protocolos de Conocimiento Cero

Lucía Martínez Martínez

Grado en Ingeniería Informática
Universidad Complutense de Madrid



Trabajo de Fin de Grado
Curso 2025 - 2026

Directores:
Clara Rodríguez Núñez
Alejandro Hernández Cerezo

Índice general

1. Introducción a Circom	4
1.1. Zero Knowledge Proof	4
1.2. CIRCOM	5
1.3. R1CS	7
1.4. Snarkjs	9
1.5. Preámbulo del problema	9
1.6. Objetivos y Estructura del trabajo	11
1.6.1. Objetivos	11
1.6.2. Estructura del trabajo	11
2. Definición del problema	13
2.1. Descripción del problema	13
2.2. Ejemplo: Piedra, Papel y Tijera	15
3. Desarrollo del problema	19
3.1. Traducción	19
3.2. Reglas de transformación	19
3.2.1. Operadores Lógicos	20
3.2.1.1. Traducción de operadores	20
3.2.2. Operadores de Comparación	20
3.2.2.1. Tratamiento de número negativos	20
3.2.2.2. Tamaño de los números comparados	22
3.2.2.3. Operadores $<, \leq, >$ y $\geq$	23
3.2.2.4. Operadores $==$ y $!=$	24
3.2.2.5. Traducción de operadores	26
3.2.3. Operadores Aritméticos	29
3.2.3.1. Suma, resta y multiplicación	29
3.2.3.2. División	29
3.2.3.3. Módulo	30
3.2.3.4. Traducción de operadores	30
3.2.4. Operadores de desplazamiento	30
3.2.4.1. Desplazamiento a la izquierda	31
3.2.4.2. Desplazamiento a la derecha	31
3.2.4.3. Traducción de operadores	32
3.2.5. Expresiones condicionales	32
3.2.5.1. Caso base: if-else	33

3.2.5.2.	Caso inductivo: Condicionales anidados	33
3.2.5.3.	Traducción de operadores	35
3.2.6.	Acceso a Arrays	35
3.3.	Ejemplo con constraints generadas automáticamente	35
4.	Verificación de los operadores	36
5.	Implementación	37
5.1.	Explicación y ejemplo de la construcción de las reglas	37
6.	Experimentos	38
6.1.	Aplicación de la generación automática sobre juegos simples	38
6.2.	Aplicación de la generación automática sobre circomLib y comparación de constraints	38
6.3.	Caso de uso, verificación y búsqueda de bugs	38

Introducción a Circom

1.1. Zero Knowledge Proof

En términos criptográficos, una prueba de conocimiento cero, conocida como Zero Knowledge Proof (o sus siglas inglesas ZKP), es una **familia de protocolos** que permite el establecimiento de métodos cuyo objetivo es permitir que una parte (**prover**) demuestre a otra (**verifier**) la validez de una afirmación **sin revelar información sobre la misma**, más allá de su validez [1].

A continuación se presenta un ejemplo para ilustrar este concepto:

Alejandro (prover) tiene el décimo de la lotería de Navidad que ha sido premiado y quiere demostrarle a **Clara (verifier)** que posee el número ganador, pero sin que ésta sea conocedora del mismo (y así evitar que pueda cobrar el premio o se lo robe). Para ello, utilizan una caja fuerte que solo se abre introduciendo la combinación del decimo premiado.

Clara escribe en un papel un dígito aleatorio, lo introduce en la caja y la cierra. Posteriormente, Clara se gira, de forma que no ve nada más de la caja.

Alejandro, que conoce el número del décimo, introduce la combinación, abriendo así la caja y leyendo el número escrito por Clara en el papel en voz alta. En ese momento, Clara sabe que ha podido abrir la caja, y por tanto, obtiene una prueba de que Alejandro es conocedor de esa información.

Sin embargo, Clara podría sospechar que Alejandro ha adivinado el dígito al azar, ya que la probabilidad es del 10 %, por lo que Clara decide repetir la prueba varias veces.

Si repetimos la prueba una segunda vez ($0,1 * 0,1 = 0,01$), se reduce la probabilidad inicial de 0,1 a 0,01. Es decir, a medida que las repeticiones aumentan, menor es la probabilidad de que se acierte al azar, ya que ésta converge a 0, alcanzando así una certeza práctica sin haber revelado ni un solo dígito del décimo.

Ejemplo 1.1: Zero Knowledge Proof

Como hemos visto en el *Ejemplo 1.1*, el concepto de **ZKP** queda ilustrado de una forma muy intuitiva. Sin embargo, para trasladar esta lógica al contexto criptográ-

fico, es necesario formalizar estas interacciones mediante protocolos específicos.

La **criptografía** constituye el pilar fundamental de la seguridad en el mundo digital, permitiendo proteger información sensible frente a amenazas. Las técnicas criptográficas nos ofrecen la capacidad de garantizar confidencialidad así como un equilibrio entre transparencia y privacidad.

Históricamente, el término de ZKP apareció en los años 80 como un concepto puramente teórico, hasta su desarrollo en la práctica [2]. Dicha noción surgió con la intención de reforzar tanto la seguridad como la privacidad en el ámbito criptográfico. Actualmente, esto ha llevado a la creación de grandes familias de protocolos, destacando especialmente **zk-SNARKs**.

Definición 1 (zk-SNARKS) *Succinct Non-Interactive Argument of Knowledge, mayormente conocido por sus siglas **zk-SNARKs**, son pruebas criptográficas las cuales permiten probar la veracidad de información sin revelar el contenido de la misma. [3].*

Estas pruebas destacan por su reducido tamaño de prueba y corto tiempo de verificación. Sin embargo, presentan la desventaja de requerir una fase inicial conocida como **configuración de confianza**.

Definición 2 (Configuración de confianza) *Una configuración de confianza es una fase en la que se produce la generación de valores aleatorios que deben destruirse de forma inmediata. Si estos valores aleatorios son expuestos, se compromete la seguridad del sistema [3].*

El **motivo** de que estos número aleatorios deban borrarse inmediatamente es debido a que cualquier entidad conocedora de esos valores podría almacenarlos y así generar **pruebas falsas**, de forma que se podría vulnerar la seguridad del sistema.

Dada la complejidad que implica implementar estos protocolos desde cero, se han desarrollado lenguajes de programación específicos denominados **Domain Specific Languages**, conocidos como DSL.

El objetivo es que los desarrolladores que no son expertos en criptografía puedan programar las declaraciones que desean demostrar. Es en este contexto donde cobra importancia **Circom**, una herramienta creada con el fin de facilitar la creación de circuitos y su posterior transformación en sistemas de restricciones.

1.2. CIRCOM

Circom es un lenguaje de programación basado en la descripción de circuitos (DSL), cuyo objetivo principal no es el cálculo, sino definir un sistema de ecuaciones que describe las relaciones entre las **señales** de entrada y salida [4].

Una señal es un componente que actúa como cable dentro del circuito, el cual transporta valores a través de las puertas lógicas del mismo y que forma parte de las

ecuaciones que el programador desea verificar. Las restricciones del sistema de ecuaciones generado por Circom describen el comportamiento de las señales del circuito.

Esta visión es fundamental, ya que las ZKP no pueden interpretar código directamente; requieren que la computación se exprese en una estructura de restricciones algebraicas denominadas R1CS, que detallaremos en la **Sección 1.3**.

El proceso de compilación de Circom refleja un paradigma híbrido entre lo **declarativo e imperativo**. De esta forma, el compilador genera dos archivos esenciales para generar la prueba:

- **Archivo R1CS:** Contiene el conjunto de todas las restricciones algebraicas que definen la lógica del circuito. Estas restricciones son añadidas desde un programa Circom por medio de los símbolos `<==` y `===`.
- **Generador de Testigo (Witness):** Un ejecutable (en formato C++ o WASM) encargado de calcular los valores de todas las señales del circuito, incluyendo los valores intermedios, a partir de las entradas proporcionadas. Estas asignaciones son añadidas desde un programa Circom por medio de los símbolos `<==` y `<--`.

Partiendo de estos dos archivos, podemos generar la prueba por medio de **Snarkjs**. Esta herramienta toma el archivo R1CS y el Witness obtenidos tras la compilación de un programa Circom para generar la prueba asociada. Analizaremos en detalle su funcionamiento y comandos específicos en la **Sección 1.4**.

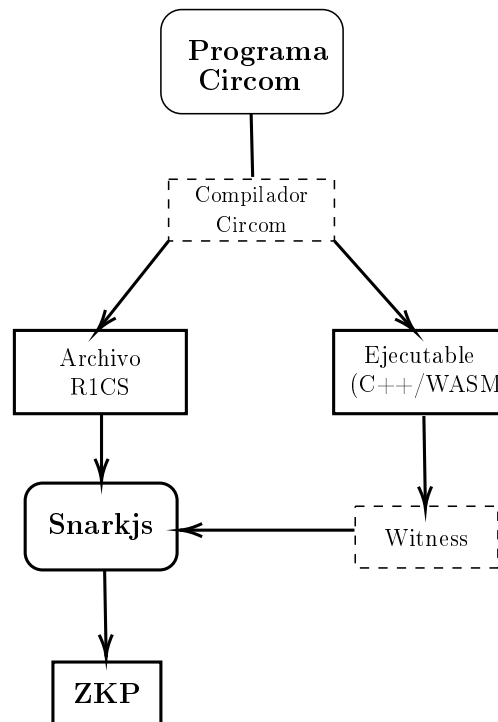


Figura 1.1: Proceso de transformación a ZKP desde Circom

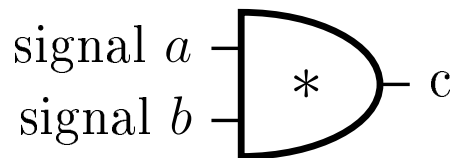
Una característica fundamental de Circom es que es un lenguaje basado en circuitos, donde la estructura del mismo debe ser estática; esto implica que cualquier valor determinante en la estructura del circuito (como tamaños de arrays, número de iteraciones de un bucle, etc..) **debe conocerse en tiempo de compilación**. Esto se debe a que el archivo R1CS contiene las restricciones algebraicas que describen el circuito, las cuales no pueden depender de los valores de las señales de entrada.

Como hemos mencionado anteriormente, las señales son las restricciones establecidas por el programador, las cuáles quiere verificar. A continuación se muestra un ejemplo de un programa Circom y su representación en un circuito lógico, en el que se añade la restricción $a * b = c$.

```

1      template mult(){
2          signal input a;
3          signal input b;
4          signal output c;
5          c <== a * b;
6      }
7
8      main component = mult();

```



Ejemplo 1.2: Ilustración comparativa código-circuito

El ejemplo anterior refleja como Circom transforma la ecuación $c <== a * b$ en un circuito lógico equivalente. De esta forma podemos ver que las entradas del componente lógico son a y b, forzando de esta manera a que c sea igual al producto de ambas entradas.

1.3. R1CS

Rank-1 Constraint System (conocido como **R1CS**), es un sistema de representación matemático diseñado para transformar tareas computacionales complejas en un conjunto de restricciones algebraicas. Su relevancia incide en la propiedad de que generar la solución a un problema puede ser caro, mientras que su **verificación es muy eficiente y rápida**, clave en las pruebas de conocimiento cero [5].

R1CS impone restricciones matemáticas complejas. Las ecuaciones, aparte de no poder exceder el segundo grado (*condición necesaria, pero no suficiente*), tienen que poder expresarse como el producto de dos combinaciones lineales de variables, teniendo como resultado una tercera combinación lineal:

$$A \cdot B - C = 0$$

Donde A , B y C son combinaciones lineales de las señales del circuito. En términos prácticos, esto implica que una sola restricción solo puede contener una multiplicación entre variables. Operaciones que incluyan múltiples productos independientes, aunque sean de segundo grado, deben ser descompuestas en varias restricciones consecutivas por medio de variables auxiliares con el fin de cumplir con este estándar.

Siguiendo el *Ejemplo 1.2*, la restricción $c \leq a \cdot b$ se traduce al formato R1CS como el producto de dos combinaciones lineales A y B , forzando así que el valor de la suma de las señales a y b sea c .

Dada la declaración:

$$(a) \times (b) = (c)$$

Donde la estructura $A \cdot B = C$ se satisface asignando:

- $A = a$ (Combinación lineal 1)
- $B = b$ (Combinación lineal 2)
- $C = c$ (Resultado de la restricción)

Ejemplo 1.3: Transformación de `var_zero.circom` a formato R1CS

Gracias a utilizar el formato R1CS, *las restricciones escritas en Circom se pueden verificar de forma sumamente eficiente* a través de Snarkjs, como veremos en la **Sección 1.4**.

1.4. Snarkjs

Snarkjs es una librería de JavaScript creada para la generación y verificación de pruebas de conocimiento cero partiendo de las restricciones en formato R1CS correspondientes, que puede ser ejecutado tanto en dispositivos móviles como en servidores con *Node.js*, materializando el concepto de *1 zk-SNARKS* nombrado anteriormente [4][6].

Como se mostró previamente en la *Figura 1.1*, Snarkjs requiere de dos archivos para generar la prueba:

- *Witness*, generado por el ejecutable C++/WASM.
- *Archivo R1CS*, restricciones de la lógica del circuito.

Una vez disponibles estos dos archivos y la configuración de confianza finalizada, Snarkjs genera la prueba de conocimiento cero que permite que el Prover pueda demostrarle al Verifier la validez de su declaración.

1.5. Preámbulo del problema

Circom es un lenguaje de programación cuyo aprendizaje y aplicación puede ser complejo inicialmente, sobre todo si se carece de habilidad tratando con lenguajes lógicos basados en restricciones. Una de las razones principales de esta complejidad son los conocidos como *underconstrained bugs*.

Definición 3 (Underconstrained bug) *Un underconstrained bug es un error producido cuando el usuario no introduce todas las ecuaciones necesarias en el fichero R1CS para reflejar el comportamiento del circuito.*

Un programa Circom es correcto cuando las ecuaciones algebraicas del archivo R1CS representan exactamente el comportamiento del circuito (expresado de forma imperativa en el generador de witness). La ausencia de las restricciones conlleva que pueden generarse witnesses que no reflejan el correcto comportamiento del circuito, pero que se aceptarían al verificar las ecuaciones del R1CS.

El símbolo `<--` permite añadir una asignación al archivo Witness, pero no al R1CS, ya que este es exclusivo para las restricciones del programa (añadidas con los operadores `<==` y `==`).

La funcionalidad de estos operadores es completamente diferente. Al principio, muchas personas que comienzan a programar en Circom no tienen clara la diferencia real entre estos operadores, desarrollando programas erróneos cuya prueba generada no es válida.

Un uso incorrecto de estos operadores podría provocar problemas como el mencionado underconstrained bug, de forma que si el usuario utiliza `<--` para añadir restricciones al sistema de ecuaciones en lugar de `<==` o `==`, habrá ecuaciones que

no serán añadidas al archivo R1CS, y por tanto, no formarán parte del sistema de ecuaciones que describe el circuito, quedando incompleto.

Un ejemplo de este tipo de error es el siguiente bloque de código, cuyo objetivo es el de comprobar si una entrada es igual a cero:

```
1   template isZero() {
2       signal input a;
3       signal output b;
4
5       signal inv;
6
7       inv <-- a!= 0 ? 1/a : 0;
8
9       b <== -a*inv * 1;
10  }
```

Código 1.1: Underconstrained bug

Esta función, en apariencia correctamente implementada, presenta un mal uso del operador `<--`, provocando el mencionado underconstrained bug. Al utilizar este operador para el cálculo de la señal `inv`, se realiza una asignación pero no se añade una restricción al sistema de ecuaciones.

De esta forma, al no haber ninguna restricción que obligue a que `inv` sea el inverso de `a`, el vínculo entre la entrada `a` con la salida `b` no se cumple totalmente. Por tanto, al no ser añadida la asignación `inv <-- a!= 0 ? 1/a : 0` al sistema de ecuaciones, no existe para el verificador, permitiendo que un prover malicioso pueda manipular el valor de `inv` para generar pruebas falsas que el sistema aceptaría.

Más allá del ejemplo expuesto, este error se repite de forma recurrente en muchos programas Circom, sobre todo aquellos desarrollados por principiantes en el lenguaje. Otros ejemplos donde este error está presente son en implementaciones de terceros que pueden encontrarse en repositorios públicos como el juego de "*Hundir la flota*" [7], *Dark Forest* [8] o *Decoder template en Circomlib* [9].

Debido a este error, entre muchos otros, Circom es un lenguaje complejo, al que cuesta adaptarse, en gran parte por los diferentes conceptos a similar. Por ello, el objetivo de este trabajo es facilitar al usuario una forma de programar en Circom muchas más rápida y sencilla, así como segura.

Aprovechando la flexibilidad de las asignaciones y la sintaxis ofrecida por la naturaleza *declarativa-imperativa* de Circom, podemos implementar una nueva funcionalidad realizando modificaciones en su compilador, permitiendo así traducir la lógica imperativa (restricciones declaradas con `<--`, únicamente añadidas al *Witness*) en restricciones que formarán parte del sistema de ecuaciones (únicamente añadidas por medio de `<==`).

De este modo, **el compilador actuaría como un puente**, transformando automáticamente el flujo imperativo en un sistema de restricciones.

Dicha transformación se consigue mediante la traducción de las reglas existentes en el flujo imperativo de Circom. Este proceso consiste en analizar los recursos existentes y especificar una regla de traducción para cada uno de ellos. De esta forma, el compilador consigue transformar automáticamente lo implementado en un sistema de restricciones.

1.6. Objetivos y Estructura del trabajo

1.6.1. Objetivos

El objetivo principal de este trabajo es crear una manera más sencilla e intuitiva de programar en Circom, consiguiendo que la creación de sistemas de ecuaciones y sus circuitos lógicos equivalentes sea mucho más accesible y segura.

De manera complementaria, el trabajo busca obtener una herramienta de validación. Aprovechando las nuevas capacidades del compilador, permitir al usuario comparar su implementación en Circom con la generada mediante la nueva funcionalidad, facilitando la comprobación entre su lógica y la generada por el compilador.

Este último objetivo también incluye el aprendizaje, buscando reducir la complejidad inicial de Circom, permitiendo al usuario familiarizarse con el lenguaje de una forma más sencilla y eficiente.

Para alcanzar los objetivos propuestos, hemos realizado las siguientes tareas durante el desarrollo del trabajo:

1. Diseño y especificación de las reglas de traducción
2. Implementación dentro del compilador de las reglas especificadas
3. Verificación de los sistemas de ecuaciones generados por las reglas
4. Aplicación de la nueva funcionalidad en casos reales
5. Desarrollo de la memoria para recoger todo lo anterior y analizar los resultados del trabajo

1.6.2. Estructura del trabajo

Para alcanzar los objetivos propuestos, este documento se ha estructurado en los siguientes capítulos:

- Capítulo 1 (Introducción a Circom): Se introduce el contexto sobre la tecnología que vamos a tratar y ofrecer una visión general de lo que pretendemos hacer con ella a lo largo del trabajo.
- Capítulo 2 (Definición del problema): Se definirá el problema y se presentará un ejemplo.
- Capítulo 3 (Desarrollo del problema): Especificación técnica de las modificaciones que se llevarán a cabo en el compilador del lenguaje.

- Capítulo 4 (Implementación):
- Capítulo 5 (Experimentos):

Definición del problema

2.1. Descripción del problema

La problemática principal que surge durante el proceso de aprendizaje de Circom reside sustancialmente en su diferencia respecto a lenguajes de alto nivel, tanto en objetivo como en restricciones de uso.

Circom, como se ha mencionado previamente, es un lenguaje de bajo nivel, presentando dificultades en su aprendizaje debidas al gran cambio de paradigma respecto a otros lenguajes de programación, más comunes entre los usuarios.

El cambio a un lenguaje basado en circuitos requiere una conversión hacia otra mentalidad, tanto para comprender cuál es el objetivo real de su naturaleza basada en circuitos, así como adaptarse a los recursos ofrecidos por el lenguaje.

Por tanto, se presenta un doble reto:

1. **Asimilar el paradigma de verificación frente al de ejecución**, comprender que el objetivo principal no es realizar un cálculo o cómputo, sino generar un sistema de restricciones con el objetivo de verificar la validez de una demostración.
2. **Adaptarse a la complejidad del diseño algebraico**, procedimiento que implica crear circuitos en los que cada operación tiene que ser expresada a través de restricciones algebraicas lineales, asegurando de esta manera el cumplimiento del formato R1CS y la descomposición de la lógica algebraica.

De esta manera, puede observarse que Circom es un lenguaje que presenta una curva de aprendizaje elevada, de forma que requiere de tiempo y dedicación para desarrollar habilidad y destreza.

Por tanto, para aquellas personas que requieran el uso de este lenguaje, pero se enfrenten a la dificultad del cambio de paradigma se expone una solución cuyo objetivo es facilitar el uso de Circom, explotando su cualidad imperativa.

La propuesta radica en la modificación del compilador de Circom para incorporar una nueva funcionalidad que habilite la traducción del cuerpo imperativo desarrollado en un programa Circom a ecuaciones algebraicas en formato R1CS equivalentes.

El cuerpo imperativo tras la modificación del compilador no sería únicamente funcional para la generación del testigo, sino que con dicha propuesta se permitiría la conversión del código correspondiente tal y como se haría si dicho cuerpo fuera el equivalente al generador de las restricciones R1CS.

Ejemplo 2.1: Implicaciones de la nueva funcionalidad

Compilador sin modificación	Compilador con modificación
<pre> template funcion() { signal input a; signal output b; signal inv; inv <-- a!=0 ? 1/a : 0; b <== -a*inv +1; a*b == 0; } </pre>	<pre> template funcion() autocomplete { signal input a; signal output b; if(a==0){ b <-- 1; } else{ b <-- 0; } } </pre>

Los códigos expuestos en *Ejemplo 2.1* son equivalentes. El objetivo de ambos códigos es construir un sistema de ecuaciones para determinar si una entrada cualquiera es igual a cero. La diferencia radica en que el primero ha sido implementado utilizando las características que ofrece Circom, mientras que el segundo se ha implementado utilizando las modificaciones realizadas en este trabajo sobre el compilador del lenguaje.

El primer bloque de código, a pesar de no hacer uso de la nueva funcionalidad, no genera un underconstrained bug aunque utilice el operador `<--`. Aludiendo al *Código 1.1*, puede apreciarse que se diferencian exclusivamente en la restricción `a*b==0`.

Esta restricción permite vincular completamente la entrada `a` con la salida `b`. Su ausencia en el código 1.1 es lo que provoca el mencionado underconstrained bug. Al añadirla, se soluciona el error, ya que, a pesar de que `inv` podría adoptar un valor distinto al que se le asignó, la señal `a` queda ahora limitada por el valor de `b` a través de la restricción `a*b==0`, la cual sí se incorpora al sistema de ecuaciones. De este modo, existe una restricción que obliga a la señal `a` a adoptar un único valor, garantizando así un circuito seguro.

Por otro lado, el segundo bloque de código es equivalente al sistema de ecuaciones generado por el primer bloque de código, salvo que este último aprovecha la nueva funcionalidad del compilador.

A través del análisis del código de este segundo bloque, se distingue la sencillez de un bloque básico `if-else` que tiene como objetivo determinar si la entrada `a` es igual a cero o no.

Puede apreciarse que la lógica necesaria para implementar una función tan primaria sin las modificaciones se convierte en una tarea más tediosa, mientras que con las

modificaciones esta lógica se convierte en un simple bloque condicional que cualquier persona familiarizada con la programación sabría hacer.

De esta forma, Circom sería un lenguaje de programación mucho mas accesible para los usuarios, al facilitar su uso incorporando esta nueva funcionalidad.

Para ejemplificar de manera práctica las restricciones mencionadas y la dificultad que supone definir manualmente limitaciones más allá del *Ejemplo 2.1*, se presenta a continuación un ejemplo que contrasta el flujo actual de Circom frente a la solución propuesta.

2.2. Ejemplo: Piedra, Papel y Tijera

Una manera más precisa de materializar esta propuesta es por medio de un ejemplo, utilizando para ello un programa real en Circom.

El programa que utilizaremos como ejemplo es el clásico juego de *piedra, papel y tijera*. Éste consiste en 3 reglas principales.

1. Piedra gana a Tijera, pero pierde contra Papel
2. Papel gana a Piedra, pero pierde contra Tijera.
3. Tijera gana contra Papel, pero pierde contra Piedra.

Este juego tan característico y conocido se debe en gran parte a su simplicidad. Sin embargo, transformar su lógica en un programa Circom que lo materialice en un circuito es una tarea más tediosa y compleja.

Con el fin de aclarar esta transición entre la lógica del juego y su implementación técnica, se ha desarrollado un repositorio específico que alberga el código fuente del circuito, implementado tanto con la nueva funcionalidad del compilador como sin ella [10]. A continuación, se muestra un fragmento de este programa Circom equivalente a la lógica del juego:

```

template piedra_papel_tijera
() {
    signal input jugador1;
    signal input jugador2;
    signal output ganador;

    signal eq; signal inv;
    signal out; signal j1;
    signal j2;

    // Jugador 1
    component J1_0 = igualdad
        (0);
    J1_0.in <== jugador1;
    component J1_1 = igualdad
        (1);
    J1_1.in <== jugador1;
    component J1_2 = igualdad
        (2);
    J1_2.in <== jugador1;

    // Jugador 2
    component J2_0 = igualdad
        (0);
    J2_0.in <== jugador2;
    component J2_1 = igualdad
        (1);
    J2_1.in <== jugador2;
    component J2_2 = igualdad
        (2);
    J2_2.in <== jugador2;

    // Igualdad (empate)
    signal eq0; signal eq1;
    signal eq2;
    eq0 <== (J1_0.out)*(J2_0.
        out);
    eq1 <== (J1_1.out)*(J2_1.
        out);
    eq2 <== (J1_2.out)*(J2_2.
        out);
    eq <== eq0 + eq1 + eq2;

    // Inversion de empate
    inv <== (1 - eq);

    // Ganadoras Jugador 1
    signal j1_0; signal j1_1;
    signal j1_2;
    j1_0 <== (J1_0.out)*(J2_2.
        out);
    j1_1 <== (J1_1.out)*(J2_0.
        out);
    j1_2 <== (J1_2.out)*(J2_1.
        out);
    j1 <== j1_0+j1_1+j1_2;

    // Ganadoras Jugador 2
    signal j2_0; signal j2_1;
    signal j2_2;
    j2_0 <== (J1_0.out)*(J2_1.
        out);
    j2_1 <== (J1_1.out)*(J2_2.
        out);
    j2_2 <== (J1_2.out)*(J2_0.
        out);
    j2 <== j2_0+j2_1+j2_2;

    // Calculo final
    out <== j2;
    ganador <== ((1-inv)*2)+(
        out*inv);
}

```

Figura 2.1: Implementación de Piedra, papel y tijera

Tras la introducción del código equivalente a la lógica del juego, puede apreciarse a simple vista la complejidad de describir flujos lógicos usando Circom, requiriendo tanto de maestría en su sintaxis y recursos, como de destreza para reflejar fielmente la lógica del juego.

Sin embargo, la incorporación de nuestra modificación sobre el compilador simplificará este proceso, permitiendo una programación más intuitiva sin privar al lenguaje de las restricciones algebraicas necesarias para generar la prueba de conocimiento cero.

Dicha incorporación simplifica considerablemente el flujo lógico a describir, ya que se permite más libertad en cuanto al uso de recursos. A continuación, se presenta una comparativa que ilustra cómo la nueva funcionalidad simplifica operaciones que, en Circom estándar, requieren un diseño aritmético.

Ejemplo 2.2: Traducción de la lógica de empate

Circom Estándar	Modificación (autocomplete)
<pre> signal eq0, eq1, eq2; eq0 <== (J1_0.out) * (J2_0. out); eq1 <== (J1_1.out) * (J2_1. out); eq2 <== (J1_2.out) * (J2_2. out); eq <== eq0 + eq1 + eq2; inv <== (1 - eq); </pre>	<pre> if(jugador1 == jugador2){ ganador <-- 2; } </pre>

Como se observa, incluso en una comprobación de igualdad, el usuario debe gestionar señales auxiliares e inversas (*inv*). Esta complejidad aumenta exponencialmente al definir las condiciones de victoria del jugador 1, en el **Ejemplo 2.3**.

Ejemplo 2.3: Traducción de la lógica del jugador 1

Circom Estándar	Modificación (autocomplete)
<pre> signal j1_0; signal j1_1; signal j1_2; j1_0 <== (J1_0.out) * (J2_2. out); j1_1 <== (J1_1.out) * (J2_0. out); j1_2 <== (J1_2.out) * (J2_1. out); j1 <== j1_0 + j1_1 + j1_2; </pre>	<pre> else if(jugador1 == 0 && jugador2 == 2){ ganador <-- 0; } else if(jugador1 == 1 && jugador2 == 0){ ganador <-- 0; } else if(jugador1 == 2 && jugador2 == 1){ ganador <-- 0; } </pre>

En estos bloques de código equivalentes, uno está íntegramente escrito con el operador `<==` para incorporar todas las restricciones al sistema de ecuaciones. El otro, a pesar de no utilizar este operador, puede usar el operador `<--` sin inconvenientes debido a las modificaciones que hemos hecho sobre el compilador; de este modo, las restricciones se añaden también al archivo R1CS y no solo al testigo.

Por lo tanto, puede apreciarse que el esfuerzo lógico para describir la lógica del jue-

go utilizando el operador $\leq$ es mucho más complejo que aprovechando la nueva funcionalidad añadida. Su uso evita que el programador tenga que transformar manualmente cada bifurcación del algoritmo en expresiones algebraicas, delegando ese trabajo al compilador modificado.

Tras analizar ambos bloques de código lógicamente equivalentes, es evidente la simplicidad, legibilidad e independencia que la nueva funcionalidad incorpora al lenguaje.

Si bien este beneficio es ya notable en un ejemplo sencillo como este, su verdadero potencial reside en la migración a problemas más complejos.

Al trasladar esta lógica a sistemas de mayor envergadura, los beneficios en términos de mantenibilidad y reducción de errores en el diseño de circuitos como una mayor accesibilidad pueden ser exponenciales.

Desarrollo del problema

3.1. Traducción

A lo largo del documento estudiaremos cómo traducir instrucciones y expresiones generadas por un lenguaje imperativo simple en un conjunto de ecuaciones en formato R1CS equivalente definimos las siguientes funciones:

Definición 4 (Traducción de expresiones) *Definimos la función de traducción de expresiones $TC : Expr \rightarrow C \times Expr$ siendo:*

- *C un sistema de ecuación en formato R1CS*
- *C es una expresión lineal*

La intuición es que en caso de que las constraints C se verifiquen, entonces e y e' siempre tienen el mismo valor.

Vamos a definir la función TC de forma inductiva, comenzando por los casos base (números y variables), para luego pasar a definir los casos compuestos.

Casos base

- Si la expresión es un número n :

$$TC(n) = (TC(\emptyset, n))$$

- Si la expresión es una variable v :

$$TC(v) = (TC(\emptyset, v))$$

3.2. Reglas de transformación

Esta sección recoge la documentación sobre la implementación de las reglas de transformación pertinentes para materializar la funcionalidad descrita a la práctica.

Las reglas de transformación especificadas abarcan los operadores lógicos, de comparación, aritméticos, de desplazamiento además del acceso a arrays y las expresiones condicionales.

3.2.1. Operadores Lógicos

Los operadores lógicos se implementan por medio de la reproducción de las puertas lógicas *AND*, *OR* y *NOT*.

Dicha lógica se aplica de la misma forma a los operadores lógicos bit a bit, con la diferencia de que cada operación se realiza sobre cada par de bits.

3.2.1.1. Traducción de operadores

Operación	Regla de Inferencia
$\text{not } e_1$	$\frac{TC(e_1) = (C_1, e'_1)}{TC(\text{not } e_1) = (C_1 \wedge \{e'_1 \cdot (e'_1 - 1) == 0\}, 1 - e'_1)}$
$e_1 \text{ and } e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{ and } e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} e'_1 \cdot (e'_1 - 1) == 0, \\ e'_2 \cdot (e'_2 - 1) == 0, \\ v_{aux} = e'_1 \cdot e'_2 \end{array} \right\}, v_{aux} \right)}$
$e_1 \text{ or } e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{ or } e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} e'_1 \cdot (e'_1 - 1) == 0, \\ e'_2 \cdot (e'_2 - 1) == 0, \\ v_{aux1} = e'_1 \cdot e'_2, \\ v_{aux} == e'_1 + e'_2 - v_{aux1}, \end{array} \right\}, v_{aux} \right)}$
$e_1 \& e_2$	$\frac{TC(e_1) = (C_1, e'_{1,bin}) \quad TC(e_2) = (C_2, e'_{2,bin})}{TC(e_1 \& e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \forall i \in [0, n-1] : \\ v_i == e'_{1,bin}[i] \cdot e'_{2,bin}[i] \end{array} \right\}, \vec{v} \right)}$
$e_1 e_2$	$\frac{TC(e_1) = (C_1, e'_{1,bin}) \quad TC(e_2) = (C_2, e'_{2,bin})}{TC(e_1 e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \forall i \in [0, n-1] : \\ v_{aux_i} == e'_{1,bin}[i] \cdot e'_{2,bin}[i], \\ v_i == e'_{1,bin}[i] + e'_{2,bin}[i] \cdot v_{aux_i} \end{array} \right\}, \vec{v} \right)}$

3.2.2. Operadores de Comparación

3.2.2.1. Tratamiento de número negativos

Un detalle destacable en la función *LessThanEq()*, utilizada para hacer posible la traducción de los operadores de comparación $<, \leq, >$ y $\geq$ a restricciones algebrai-

cas, es que para números negativos podría dificultar su uso si no se tienen en cuenta ciertos aspectos determinantes.

El motivo reside en trabajamos sobre un cuerpo finito $\mathbb{F}_p$, de forma que los números negativos realmente no existen. En su lugar, se representan mediante el módulo del campo.

Por ejemplo, el valor -1 se interpreta como $p-1$, un número extremadamente grande que se aproxima al orden del primo del campo.

Para solventar este problema, recurrimos a la transformación de los números a comparar de sistema decimal a binario. Esta conversión es estrictamente necesaria para asegurar que los números correspondientes no exceden el rango especificado.

La función $Num2Bits(n)$ consigue cumplir esta precondition ya que su funcionamiento radica en la transformación numérica de decimal a binario, reconstruyendo el número decimal entrante durante la misma.

De esta forma, al terminar la transformación y antes de salir de la función, se realiza una comprobación para verificar si el número decimal entrante y el número decimal reconstruido durante la transformación son iguales.

Debido al excesivo tamaño del número entrante, nunca se cumplirá la restricción de que ambos sean iguales debido a que el tamaño máximo fijado (N) es siempre menor que el tamaño del número negativo, desencadenando que la reconstrucción no realizará un bucle de tamaño mayor a al número N fijado.

Por tanto, no habrá un bucle que reconstruya un número decimal negativo ya que nunca se realizan las iteraciones necesarias, forzando de esta forma a que solo los números positivos cumplan dicha condición.

Intuición del funcionamiento de $Num2Bits(n)$ con un número negativo entrante:

1. El número negativo se interpreta como el módulo del campo.
2. Éste se interpretará como un número superior a $2^n - 1$, saliéndose fuera del rango $[0, 2^n - 1]$, por tanto la restricción $lc1 === in$ de la función nunca se cumplirá en este caso y la prueba no se generará.

En conclusión, $Num2Bits()$ garantiza el uso de número positivos dentro del rango mencionado de manera implícita.

Observación: Gracias a la constraint $lc1 === in$ (fuerza a que la entrada cumpla el rango $[0, 2^n - 1]$), podemos evitar el uso de número negativos, ya que estos no cumplirán tal restricción.

Num2Bits(n):

```
1      template Num2Bits(n){
2          signal input in;
3          signal output out[n];
4          var lc1=0; // Reconstruye el numero decimal a
                    // convertir para la constraint
5
6          var e2=1; // Evitar la potencia
7          for (var i = 0; i<n; i++) {
8              out[i] <-- (in >> i) & 1; // Contruimos el numero
                    // en binario
9              out[i] * (out[i] -1 ) == 0;
10             lc1 += out[i] * e2;
11             e2 = e2+e2;
12         }
13
14         lc1 == in; // Constraint mencionada anteriormente
15     }
```

Código 3.1: Función Num2Bits(n)

Observación: Esta implementación se trata de un pseudocódigo equivalente a la implementación del sistema de ecuaciones desarrollado en el compilador.

3.2.2.2. Tamaño de los números comparados

El tamaño de los números comparados es un factor importante, ya que dicho tamaño puede ser por un lado conocido e igual, mientras que por otro lado puede ser desconocido o de diferentes tamaños.

Para resolver este problema, vamos a definir un complemento opcional por cada operador de comparación existente.

Dicho complemento consiste en restringir el tamaño de los números comparados a un tamaño especificado por el usuario. Si el usuario no hace uso de este complemento opcional para la especificación del tamaño de los números a comparar, se utilizará el tamaño máximo por defecto, correspondiente a 253.

Definición 5 (Complemento opcional de tamaño) *Vamos a definir el operador:*

Operador *op*

- *op: Comparaciones de cuyos números se conoce el tamaño y además tienen el mismo tamaño.*
- *La ausencia de dicho componente se traduce a la asunción del 253 como el tamaño fijo de ambos números comparados, el mayor tamaño posible posible.*

Operador op_n

- op_n : Comparaciones de cuyos números no se conoce el tamaño y/o no tiene el mismo tamaño. De esta manera, n indica el tamaño fijado para ambos números comparados.

Este complemento aplica para todos los operadores de comparación, siguiendo la misma estructura que la del *Ejemplo 5*;

Un caso de uso se presenta a continuación para clarificar la naturaleza de dicho complemento, así como su aplicación.

```
1      template Num2Bits(n)  autocomplete{
2          signal input  a;
3          signal output b;
4
5          if(a >_(2) b){
6              b <-- 2;
7          }
8          else if(a ==_(2) b){
9              b <-- 1;
10         }
11         else{
12             b<--0;
13         }
14     }
```

Ejemplo 3.1: Caso de uso del complemento opcional de tamaño

3.2.2.3. Operadores $<, \leq, >$ y $\geq$

Para la implementación de dichos operadores únicamente es necesaria la función *SecureLessThanEq(n)*.

El motivo reside en que una comparación entre una variable a y b , con *SecureLessThanEq(n)* podemos conocer si $a \leq b$ o bien $a > b$ en caso contrario (*GreatherThan()*).

Siguiendo esta lógica, podemos usar la misma función invirtiendo el orden de los parámetros, consiguiendo de esta manera los operadores *GreatherThanEq()* y *LessThan()* si $b \leq a$ o $b > a$.

La implementación de *SecureLessThanEq(n)*, aparte de utilizar la función *Num2Bits()* para cada par de variables a comparar, aplica Complemento a 2.

Es decir, dadas las variables A y B , previamente transformadas a bits, aplicamos $(\text{NOT } A) + B + 1$. La inversión de los bits de A y la suma tanto de B como de 1 es equivalente a la operación $B - A$.

Por tanto, cuando el minuendo es mayor o igual que el sustraendo, la suma de los bits junto con el complemento a dos produce un acarreo (overflow) en la posición n (el bit extra, bit más significativo). Por otro lado, si el minuendo es menor que el sustraendo, no se produce dicho overflow.

De esta forma sabemos que si el bit de acarreo es 1, podemos concluir que $B \geq A$, ya que al hacer la suma con Complemento a 2, tenemos que:

$$\begin{aligned}
 & (\text{NOT } A) + B + 1 \\
 & [(2^n - 1) - A] + B + 1 \\
 & 2^n - 1 - A + B + 1 \\
 & 2^n - A + B \\
 & B + (2^n - A)
 \end{aligned} \tag{3.1}$$

Como podemos ver en las ecuaciones anteriores, debido al requerimiento de $2^n - 1$ por parte de NOT A, es vital el $+1$ de $(\text{NOT } A) + B + 1$. De lo contrario el resultado de la resta estaría desplazado por una unidad al no eliminar ese -1 de NOT A.

Partiendo de $2^n + B - A$, si B es mayor o igual que A, entonces $(B-A) \geq 0$. De este modo, tendríamos que $2^n + (\text{num.positivo}) \geq 2^n$, concluyendo que si $B \geq A$ obtendríamos siempre un resultado mayor o igual a 2^n , excediendo siempre los n bits y como consecuencia, produciendo un overflow.

Siguiendo la misma lógica, si el bit de acarreo es 0 significa que A es mayor que B ya que $B-A < 0$, y de esta forma obtendríamos $2^n + (\text{num.negativo}) < 2^n$, concluyendo así que no podría producirse un overflow.

Con esta pequeña modificación, al usar para ambos número a comparar la función *Num2Bits()* nos aseguramos de que se cumpla la restricción de ser número enteros positivos dentro del rango.

3.2.2.4. Operadores == y !=

Los operadores `==` y `!=` plantean una implementación alejada del complemento a 2, estrategia empleada en los anteriores operadores. La estrategia aplicada en este caso es simplemente establecer dos restricciones claves:

- `(in1-in2) * equal_signal = 0`
- `1 - ((in1-in2) * inv_signal) = equal_signal`

Estas dos ecuaciones son imprescindibles, ya que las usaremos para determinar si `in1 == in2` o `in1 != in2`, utilizando `equal_signal` como indicador.

La señal `equal_signal` será 1 cuando `in1 == in2`, o será 0 cuando `in1 != in2`. Este comportamiento se consigue por medio de las dos ecuaciones descritas anteriormente.

`(in1-in2) * equal_signal = 0` es imprescindible para obtener que `equal_signal` sea 0 cuando `in1` e `in2` sean diferentes. Esto se debe a que si `in1` es distinto a

in2, entonces el resultado de su diferencia será distinto a 0. Por tanto, la ecuación quedaría así:

$$\begin{aligned} num \cdot equal_signal &= 0 \\ num &\in \mathbb{R} \setminus \{0\} \end{aligned} \quad (3.2)$$

De esta forma, al ser num un valor distinto de cero, la única forma de que se satisfaga la ecuación es forzar a que `equal_signal` sea obligatoriamente 0. De este modo, la señal será igual a 0, indicando que in1 e in2 son diferentes.

Observación: Si in1 e in2 fuesen iguales, entonces su diferencia sería igual a 0, por tanto `equal_aux` podría tomar cualquier valor e `(in1-in2) * equal_signal = 0` seguiría satisfaciéndose:

$$\begin{aligned} 0 \cdot equal_signal &= 0 \\ equal_signal &\in \mathbb{R} \end{aligned} \quad (3.3)$$

Por ello, es importante la segunda ecuación, ya que además de que `equal_signal` puede ser cualquier valor, podría ser 0 también, indicando erróneamente que in1 e in2 son diferentes.

Esta segunda ecuación, `1 - ((in1-in2) * inv_signal) = equal_signal`, además de asegurar que `equal_signal` sea distinta de cero, garantiza que el resultado sea estrictamente 1 siempre que in1 e in2 sean idénticos.

Esto se consigue mediante el uso de una segunda señal, `inv_signal`, la cual garantiza que `equal_signal` tome el valor 1 cuando in1 e in2 son iguales. Dado que en este escenario la diferencia entre ambas entradas es nula, cualquier valor de `inv_signal` $\in \mathbb{R}$ resultará en un producto igual a cero, derivando en el siguiente comportamiento:

$$\begin{aligned} 1 - ((in1 - in2) \cdot inv_signal) &= equal_signal \\ 1 - (0 \cdot inv_signal) &= equal_signal \\ 1 - 0 &= equal_signal \\ 1 &= equal_signal \end{aligned} \quad (3.4)$$

Como puede observarse, aunque en la primera ecuación para `in1 == in2` la señal `equal_signal` pueda tener cualquier valor, con esta segunda ecuación forzamos a que su valor sea 0, en caso de que `in1 != in2`, o bien 1, si `in1 == in2`.

En caso de que `in1 != in2`, entonces `equal_signal` sería 0, obteniendo un comportamiento diferente:

$$\begin{aligned} 1 - ((in1 - in2) \cdot inv_signal) &= equal_signal \\ 1 - ((in1 - in2) \cdot inv_signal) &= 0 \\ \text{Como } (in1 - in2) &\in \mathbb{R} \setminus \{0\} \\ inv_signal &= \frac{1}{in1 - in2} \\ 1 - 1 &= 0 \\ 0 &= 0 \end{aligned} \quad (3.5)$$

Gracias a la señal `inv_signal`, incluso cuando ambas entradas son diferentes, la restricción puede satisfacerse ajustando el valor de la señal al inverso de la diferencia, para así obtener como producto de estas el 1.

3.2.2.5. Traducción de operadores

A continuación, definiremos para cada operador su inferencia.

Operación	Regla de Inferencia
$e = e_1 == e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 == e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} EQI(253).in[0] = e'_1, \\ EQI(253).in[1] = e'_2, \\ EQI(253).out = v_{aux}, \\ v_{aux} == 1 \end{array} \right\}, 1 - v_{aux} \right)}$
$e = e_1 == _ (n) \ e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 == _ (n) e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} EQI(n).in[0] = e'_1, \\ EQI(n).in[1] = e'_2, \\ EQI(n).out = v_{aux}, \\ v_{aux} == 1 \end{array} \right\}, 1 - v_{aux} \right)}$
$e = e_1 \neq e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \neq e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} EQI.in[0] = e'_1, \\ EQI.in[1] = e'_2, \\ EQI.out = v_{aux}, \\ v_{aux} == 0 \end{array} \right\}, v_{aux} \right)}$
$e = e_1 \neq _ (n) \ e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \neq _ (n) e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} EQI(n).in[0] = e'_1, \\ EQI(n).in[1] = e'_2, \\ EQI(n).out = v_{aux}, \\ v_{aux} == 0 \end{array} \right\}, v_{aux} \right)}$
$e = e_1 < e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 < e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \{SLTEq(253).in1 = e'_2, \\ SLTEq(253).in2 = e'_1, \\ SLTEq(253).out = v_{aux} \\ v_{aux} == 0\}, \end{array} \right\}, 1 - v_{aux} \right)}$

Continúa en la siguiente página...

Operación	Regla de Inferencia (cont.)
$e = e_1 < _ (n) e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 < _ (n) e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \{SLTEq(n).in1 = e'_2, \\ SLTEq(n).in2 = e'_1, \\ SLTEq(n).out = v_{aux} \\ v_{aux} == 0\}, \end{array} \right\}, 1 - v_{aux} \right)}$
$e = e_1 \leq e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \leq e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \{SLTEq(253).in1 = e'_1, \\ SLTEq(253).in2 = e'_2, \\ SLTEq(253).out = v_{aux} \\ v_{aux} == 1\}, \end{array} \right\}, v_{aux} \right)}$
$e = e_1 \leq _ (n) e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \leq _ (n) e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \{SLTEq(n).in1 = e'_1, \\ SLTEq(n).in2 = e'_2, \\ SLTEq(n).out = v_{aux} \\ v_{aux} == 1\}, \end{array} \right\}, v_{aux} \right)}$
$e = e_1 > e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 > e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \{SLTEq(253).in1 = e'_1, \\ SLTEq(253).in2 = e'_2, \\ SLTEq(253).out = v_{aux} \\ v_{aux} == 0\}, \end{array} \right\}, 1 - v_{aux} \right)}$
$e = e_1 > _ (n) e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 > _ (n) e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \{SLTEq(n).in1 = e'_1, \\ SLTEq(n).in2 = e'_2, \\ SLTEq(n).out = v_{aux} \\ v_{aux} == 0\}, \end{array} \right\}, 1 - v_{aux} \right)}$
$e = e_1 \geq e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \geq e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \{SLTEq(253).in1 = e'_2, \\ SLTEq(253).in2 = e'_1, \\ SLTEq(253).out = v_{aux} \\ v_{aux} == 1\}, \end{array} \right\}, v_{aux} \right)}$

Continúa en la siguiente página...

Operación	Regla de Inferencia (cont.)
$e = e_1 \geq_{-(n)} e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \geq_{-(n)} e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \{SLTEq(n).in1 = e'_2, \\ SLTEq(n).in2 = e'_1, \\ SLTEq(n).out = v_{aux} \\ v_{aux} == 1\}, \end{array} \right\}, v_{aux} \right)}$

Observación: Es importante darse cuenta que podemos hacer uso de la misma función para los operadores $<, \leq, >$ y $\geq$ explotando su carácter reflexivo. Para ello, notesé que las llamadas de los operadores $\leq$ y $\geq$ se invierten.

Por otro lado, EQI denota las siglas *Equality Indicator*, haciendo referencia a una función que encargada de imitar al comportamiento explicado en la *Sección 3.2.2.4* para dos entradas.

3.2.3. Operadores Aritméticos

3.2.3.1. Suma, resta y multiplicación

Los operadores de suma y resta son los más sencillos de implementar debido a su naturaleza, ya que ninguna de estas operaciones corre el riesgo de generar una declaración que exceda el segundo grado. Por tanto, siempre cumplirán el formato R1CS y no se requerirán señales intermedias para asegurar el cumplimiento de los estándares algebraicos.

Por otro lado, esto no aplica a la multiplicación. El motivo de esta diferencia con respecto a la suma y resta se debe a que existe una incertidumbre respecto al grado de la declaración. Al tratarse de un producto, no se puede garantizar que sus operandos no provengan de otra operación cuyo grado sea cuadrático.

En consecuencia, cualquier operador que corra el riesgo de exceder el segundo grado en una declaración, y por tanto de incumplir el formato establecido por R1CS, debe utilizar señales intermedias para así cumplir dicho formato.

Por otro lado, operadores como la división y el módulo plantean una implementación más compleja, ya que para ambos operadores es necesario el desarrollo y cumplimiento del *teorema de la división*, el cual dice lo siguiente.

Definición 6 (Teorema de la división)

$$\begin{aligned}a &= b \cdot q + r \\ 0 &\leq r < b \\ b &\neq 0\end{aligned}$$

Conociendo este teorema, el cuál presenta dos condiciones necesarias, en las siguientes secciones se especificará la aplicación de este teorema para conseguir la traducción a restricciones algebraicas deseada.

3.2.3.2. División

Partiendo de la ecuación $a = b * q + r$, podemos llegar al resultado de la operación a/b forzando a que se cumpla, pero despreciando el resto (ya que se trata de división entera).

El término "forzar" hace referencia a usar una señal auxiliar, de forma que partiendo del dividendo y del divisor, el cociente sea obligado a tomar el valor que buscamos, ya que de lo contrario la ecuación no podrá satisfacerse.

Ejemplo de código en Circom:

```
1   template IntegerDivision() {
2       signal input a; // Dividendo
3       signal input b; // Divisor
4       signal output c; // Cociente q
5   }
```

```

6      var q;
7      a == b * q;
8      c <= q;
9  }
10
11  component main = IntegerDivision();

```

3.2.3.3. Módulo

Partiendo de la ecuación $a = b \cdot q + r$, podemos llegar al resultado de la operación $a \% b$ con la variable r , la que representa el resto de una división.

$$a = b \cdot q + r \quad (3.6)$$

$$r = a - (b \cdot q) \quad (3.7)$$

$$r = a - (b \cdot (a/b)) \quad (3.8)$$

3.2.3.4. Traducción de operadores

Operación	Deducción
$e = e_1 + e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 + e_2) = (C_1 \wedge C_2, e'_1 + e'_2)}$
$e = e_1 - e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 - e_2) = (C_1 \wedge C_2, e'_1 - e'_2)}$
$e = e_1 * e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 * e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} = e'_1 * e'_2\}, v_{aux})}$
$e = a/b$	$\frac{TC(a) = (C_1, a') \quad TC(b) = (C_2, b')}{TC(a/b) = (C_1 \wedge C_2 \wedge \{v_{aux} = b' * q'\}, a' = v_{aux} + r')}$
$e = a \% b$	$\frac{TC(a) = (C_1, a') \quad TC(b) = (C_2, b')}{TC(a \% b) = (C_1 \wedge C_2 \wedge \{v_{aux} = a'/b'\} \wedge \{v_{aux_2} = b * v_{aux}\}, a' - v_{aux_2})}$

3.2.4. Operadores de desplazamiento

La operación de desplazamiento implica la transformación de un número determinado al sistema binario y, después de estar en este sistema, desplazar el número binario por una cantidad específica de bits, indicada por el usuario.

Para imitar el comportamiento descrito anteriormente, haremos uso de la función Num2Bits() descrita anteriormente para obtener el número a desplazar en binario y poder aplicar las transformaciones diferentes.

Los pasos posteriores a la transformación numérica son dependientes al tipo de desplazamiento. En función de si se trata de un desplazamiento a la izquierda o a la derecha, la construcción del sistema de ecuaciones será diferente.

Observación: Hay que asegurar de que el tamaño de N del número a desplazar sea mayor o igual que el número de bits que se quieren desplazar.

3.2.4.1. Desplazamiento a la izquierda

En este tipo de operación, el objetivo es desplazar un número X de bits (especificados por el usuario) desde las posiciones menos significativas hacia las más significativas; es decir, hacia la izquierda.

Para lograr este comportamiento, los espacios que quedan vacíos en la parte menos significativa son sustituidos por 0.

Esta explicación se complementa con el siguiente ejemplo, que describe el comportamiento de un desplazamiento a la izquierda para la operación $11 \ll 2$:

Ejemplo 3.2: Desplazamiento a la izquierda

- Dado el número decimal 11, lo convertimos a binario: 00001011
- Una vez realizada la transformación a binario, desplazamos 2 bits a la izquierda: 00001011 $\rightarrow$ 00101100
- Al transformarse los últimos dos bits en 0, el objetivo es agregar n ceros al final del array, completando así la representación binaria tras el desplazamiento.

3.2.4.2. Desplazamiento a la derecha

De manera similar al desplazamiento a la izquierda, el objetivo es desplazar un número X de bits (especificados por el usuario) desde las posiciones más significativas hacia las menos significativas; es decir, hacia la derecha.

Para lograr este comportamiento, los espacios que quedan vacíos en la parte más significativa son sustituidos por 0.

Esta explicación se complementa con el siguiente ejemplo, que describe el comportamiento de un desplazamiento a la derecha para la operación $11 \gg 2$:

- Dado un número decimal, lo convertimos a binario, en este caso 11: 00001011
- Con el número en binario, desplazamos 2 bits a la derecha: 00001011 $\rightarrow$ 00000010
- Como podemos ver, los últimos dos bits se han despreciado, añadiendo dos ceros al principio del número binario.

De la misma forma en la que se ha desarrollado estas operaciones, se definirán las ecuaciones que formarán parte del sistema dentro del compilador para reflejar ambos comportamientos.

3.2.4.3. Traducción de operadores

Operación	Deducción
$e = e_1 \ll e_2$	Para un component $izq = \text{desplazamiento_izq}(n)(a,b)$: $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \ll e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq izq.out\}, v_{aux})}$
$e = e_1 \gg e_2$	Para un component $izq = \text{desplazamiento_dcha}(n)(a,b)$: $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \gg e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq dcha.out\}, v_{aux})}$

3.2.5. Expresiones condicionales

Como se ha mencionado anteriormente, en Circom cada señal representa un cable del circuito cuyo valor es único. Por esta razón, no pueden cambiar su valor de forma dinámica una vez establecidas, ya que el sistema de restricciones debe ser estático para su correcta verificación.

Para simular decisiones condicionales basadas en entradas, se deben usar expresiones aritméticas que implementen la lógica de selección, de manera que una señal no pueda reasignar su valor.

Un ejemplo específico de conversión de un bloque condicional a sus equivalente utilizando restricciones en Circom sería el siguiente:

Ejemplo 3.3: Comparación entre bloque condicional imperativo y declarativo

Programación Imperativa	Programación Declarativa (Circom)
<pre>// Logica de control de // flujo if(entrada % 2 == 0){ printf("Es par"); } else { printf("Es impar"); }</pre>	<pre>// Definicion de // restricciones signal input entrada; signal output salida; var var_div = entrada \ 2; // Restriccion cuadratica entrada == 2 * var_div + salida; salida * (salida - 1) == 0;</pre>

El ejemplo anterior ilustra el proceso de transformación de una expresión condicional

en una expresión aritmética equivalente, evitando reasignar la señal de salida.

De la misma forma expuesta en el ejemplo, definiremos la inducción partiendo del caso base.

3.2.5.1. Caso base: if-else

Ejemplo 3.4: Caso base condicional

```
1
2      // Código en C
3      if (age < 0) {
4          printf("Edad invalida.\n");
5      } else {
6          printf("Perfil: adulto mayor.\n");
7      }

```

```
1
2      // Código en Circom
3      component less_0 = LessThan()(age,0);
4
5      signal less_0 = less_0.out;
6      signal not_less_0 = 1 - less_0.out;
7
8      mensaje <== less_0 * m1 + not_less_0 * m2;

```

El patrón de traducción identificado es el siguiente:

1. Crear una componente equivalente a la expresión lógica utilizada en el `if`
2. `if`: La restricción se activa cuando el resultado de dicho componente es igual a 1
3. `else`: La restricción se activa mediante la negación lógica de la expresión anterior

Este ejemplo representa el caso base, el cual se aplica de manera inductiva para estructuras condicionales más complejas.

La traducción formal de este caso es la siguiente:

3.2.5.2. Caso inductivo: Condicionales anidados

Partiendo del caso base, es trivial aplicar la inducción para alcanzar la transformación de bloques condicionales complejos en sistemas de ecuaciones.

A continuación, un ejemplo guiado donde aplicamos las hipótesis del caso base para llegar al resultado final.

Ejemplo 3.5: Caso inductivo condicional

```
1 // Código en C
2
3
4 if (age < 20) {
5     if(age < 10){
6         printf("Perfil: ninyo.\n");
7     }
8     else{
9         printf("Perfil: adolescente.\n");
10    }
11 } else if(age < 40){
12     if(age < 30){
13         printf("Perfil: adulto joven.\n");
14     }
15     else {
16         printf("Perfil: adulto mayor.\n");
17     }
18 }
19 else{
20     printf("Perfil: adulto mayor.\n");
21 }
```

- Por cada condición de igualdad/desigualdad se crea un componente correspondiente, como en los casos anteriores.

```
1 component less_10 = LessThan()(age,10);
2 component less_20 = LessThan()(age,20);
3 component less_30 = LessThan()(age,30);
4 component less_40 = LessThan()(age,40);
```

- Partiendo de lo citado en los apartados anteriores, tenemos que generar las condiciones excluyentes.

```
1 signal is_less_20 = less_20.out;
2 signal is_less_10 = is_less_20 * less_10.out;
3 signal isnt_less_10 = is_less_20 * (1 - less_10.out);
4
5 signal is_less_40 = less_40.out;
6 signal is_less_30 = is_less_40 * less_30.out;
7 signal isnt_less_30 = is_less_40 * (1 - less_30.out);
8
9
10 signal is_greater_40 = (1-is_less_40);
```

- Combinación ponderada de los mensajes:

```
1 mensaje <= is_less_10 * m1 + isnt_less_10 * m2 +
2 is_less_30 * m3 + isnt_less_30 * m4 +
3 is_greater_40 * m5;
```

Aplicando la inducción, puede intuirse que siempre se repite el mismo patrón en el que cada condición utiliza la negación de las condiciones anteriores y el resultado de

la comparación actual construir los bloques **if-else**

De esta manera, podemos transformas cualquier bloque condicional en un sistema de ecuaciones equivalente, cumpliendo con la naturaleza estática de Circom.

3.2.5.3. Traducción de operadores

Operación	Deducción
$\text{if } cond \text{ then } S_1 \\ \text{else } S_2$	$\frac{TC(cond) = (C_{cond}, cond') \quad TC(S_1) = (C_1, S'_1) \quad TC(S_2) = (C_S, S'_2)}{TC(\text{if } cond \text{ then } S_1 \text{ else } S_2) = (C_{cond} \wedge C_1 \wedge C_S \wedge \{v_{aux} = cond' \cdot S'_1 + (1 - cond') \cdot S'_2\}, v_{aux})}$

3.2.6. Acceso a Arrays

3.3. Ejemplo con constraints generadas automáticamente

Implementación

4.1. Explicación y ejemplo de la construcción de las reglas

Verificación de los operadores

Experimentos

- 6.1. Aplicación de la generación automática sobre juegos simples
- 6.2. Aplicación de la generación automática sobre circomLib y comparación de constraints
- 6.3. Caso de uso, verificación y búsqueda de bugs

Bibliografía

- [1] Wikipedia. (2026). *Prueba de conocimiento cero*. https://es.wikipedia.org/wiki/Prueba_de_conocimiento_cero. Accedido: 04-04-2026.
- [2] Nervos Network. (2024). *Pruebas de conocimiento cero (zero knowledge proofs)*. [https://www.nervos.org/es/knowledge-base/zero_knowledge_proofs_\(explainCKBot\)](https://www.nervos.org/es/knowledge-base/zero_knowledge_proofs_(explainCKBot)). Accedido: 02-04-2026.
- [3] Binance Academy. (2024). *ZK-SNARKs y ZK-STARKs explicados*. <https://www.binance.com/es/academy/articles/zk-snarks-and-zk-starks-explained> Accedido: 09-04-2026.
- [4] Rodríguez Nuñez, C. (2024). *Propiedades de corrección y seguridad en tecnologías blockchain*. Accedido: 04-04-2026.
- [5] Buterin, V. (2016). *Quadratic Arithmetic Programs: from Zero to Hero*. <https://vitalik.eth.limo/general/2016/12/10/qap.html>. Accedido: 04-04-2026.
- [6] Bellés-Muñoz, M., Isabel, M., Muñoz-Tapia, J. L., Rubio, A. y Baylina, J. (2023). CIRCOM: A Circuit Description Language for Building Zero-Knowledge Applications. *IEEE Software/Journal*. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10002421>. Accedido: 02-04-2026.
- [7] BattleZips Team. (2022). *BattleZips-Circom: Zero-knowledge Battleship engine* [Software]. <https://github.com/BattleZips/BattleZips-Circom>. Accedido: 03-04-2026.
- [8] Dark Forest Team (2021). *Dark Forest Circuits*. <https://github.com/darkforest-eth/circuits> Accedido: 09-04-2026.
- [9] Iden3 (2023). *Multiplexer circuit - circomlib*. Recuperado de <https://github.com/iden3/circomlib/blob/master/circuits/multiplexer.circom> Accedido: 09-04-2026.
- [10] Martínez, L. (2026). *zk-rock-paper-scissors: A Zero-Knowledge implementation in Circom* [Software]. <https://github.com/luciamarst/zk-rock-paper-scissors>. Accedido: 03-04-2026.